Search Beats Imitation: Neural-Guided Determinized MCTS for the Pokémon Trading Card Game

October 18, 2026

Antonio Cordeiro

Abstract

We build an agent for the Kaggle *Pokémon TCG AI Battle Challenge*, a stochastic, imperfect-information card game with variable action sets. A 2.6M-parameter entity/option transformer is trained on 453k expert decisions with policy, value and opponent-hand (belief) heads; its value head is re-trained on self-play outcomes, and it guides a determinized Monte Carlo tree search on the exact game simulator. Under a rigorous protocol (held-out games, a fixed six-opponent gauntlet, 95% Wilson intervals, SPRT), search raises the score against the gauntlet from 0.75 to 0.90 and the ladder rating from 650 (our previous system) to ≈770. A learned belief model predicts hidden cards better than counting (AUROC 0.75 vs 0.71) yet does not improve play: at our budgets one deep search tree beats averaging several shallow determinized trees.

1. Introduction

In the Pokémon TCG two players pilot 60-card decks; most information (hands, deck order, prize cards) is hidden, draws and coin flips are random, and the legal move set changes at every decision (play a card, attach, evolve, attack, pick targets, choose k of n cards). The Kaggle challenge provides a C++ simulator with a *search* API that can step a game forward from any observation once the hidden cards are guessed, a ladder with a Gaussian skill rating, and a budget of ≈600 s of thinking per best-of-3 match on 2 CPUs.

Our previous system stacked behavioural cloning, offline RL, PPO and weight averaging, but spent 6 ms per move and reached only 250–650 on the ladder. Its local evaluation was invalid: in 350 games it rated the 500-ladder agent *above* the 650-ladder one (score 0.32 for the latter). We therefore rebuilt the agent around (i) a simple network, (ii) search that actually uses the simulator and the time budget, and (iii) an evaluation protocol whose conclusions we can trust and check against the ladder. Code: `https://github.com/USER/ptcg-agent` (TODO).

Email: Antonio Cordeiro <antcor04@gmail.com>.

Deep Learning and Applied AI 2026, Sapienza University of Rome, 2nd semester a.y. 2025/2026.

2. Related work

AlphaGo/AlphaZero combine policy/value networks with PUCT tree search (Silver et al., 2016; 2018); expert iteration distils search back into the network (Anthony et al., 2017). For imperfect information, perfect-information Monte Carlo (PIMC) samples *determinizations* of the hidden state and searches each (Long et al., 2010); information-set MCTS shares one tree across samples (Cowling et al., 2012). Entity/pointer transformers handle variable action sets in complex games (Vinyals et al., 2019; Vaswani et al., 2017).

3. Method

Representation and network. Each observation becomes tokens: one global token (turn, prizes, deck/hand sizes, the decision type), two mean-pooled discard bags, up to 64 card tokens (Pokémon in play with HP, damage, energies; our hand; stadium; revealed cards) and up to 80 option tokens, one per legal move. An option token adds the embeddings of the card tokens it refers to (acting card, target), so the network scores moves by *what they touch*; no card-specific rules are hand-coded. A 4-layer pre-norm transformer (d=192, 2.6M parameters) mixes all tokens. Heads: a policy over option tokens (plus a count head for k-of-n choices), a value $V(s) \in [-1, 1]$, and a belief head predicting, for every card id, whether it is in the opponent's hand.

Training. We parse the 3,000 highest-rated replays (453k decisions, split 90/5/5 *by game*), pairing the observation at step s with the action recorded at step $s{+}1$. Belief labels come for free: the replay stores the opponent's own view of its hand. The loss is cross-entropy on the chosen option(s) (losers' moves weighted 0.5) plus value MSE, count BCE

Table 1. Top: offline results on held-out games. Bottom: online ablations (score of A vs B, 95% CI).

Policy top-1 (uniform-legal baseline)	70.0% (21.7%)
main phase, $\geq$5 options	60.7% (11.6%)
Value Brier: replay-only / +self-play (const.)	.254 / .226 (.249)
Belief AUROC: learned / count w/ true list	.752 / .711
MCTS-64 vs own policy (200 games)	.59 [.52, .66]
learned vs count-based hand sampling (400)	.48 [.43, .53]
4 worlds vs 1 world, equal sims (300)	.39 [.34, .45]
deployed: 1 vs 4 worlds (400)	.57 [.52, .61]
ExIt-2 + MCTS vs base + MCTS (300)	.57 [.51, .62]
ExIt-3 + MCTS vs ExIt-2 + MCTS (300)	.50 [.45, .56]

and belief BCE (12 epochs, AdamW). With only 3k game outcomes the value head overfits (test Brier worse than a constant), so we fine-tune it on 8k games of policy self-play while anchoring the policy on replay batches. Optionally, expert iteration (ExIt) adds MCTS visit distributions from 1.6k search-vs-search games as policy targets.

Search. At each single-choice decision we sample K worlds: our deck order and prizes uniformly from our unseen cards; the opponent's decklist as the library deck (120 decklists mined from replays and ladder games) most consistent with its visible cards; its hand either uniformly (count-based) or weighted by the belief head. In each world we run PUCT with network priors and value leaves on the simulator; forced moves and k-of-n choices are played by the policy. Root visits are summed over worlds. A time manager spends $\min(c, (T_{\text{left}} - r)/q)$ seconds per decision, keeping a reserve of the 600 s budget.

Evaluation. Offline metrics on held-out test games. Online: a fixed *gauntlet* of six opponents (our two previous Kaggle agents with their decks, and our imitation policy piloting four meta decks), seat-balanced, with a simulated Kaggle time budget; we report scores with 95% Wilson intervals, use paired head-to-head matches for ablations and an SPRT (H_0: +0, H_1: +30 Elo) to gate submissions.

4. Results

Offline. The policy predicts expert moves far above chance (Tab. 1); two seeds agree within 0.3 points. Self-play fixes the overfit value head. The belief head beats a count-based baseline that is even given the opponent's true decklist, without access to the decklist, presumably by learning from the public game state which cards players tend to hold.

Deck choice. Piloted by our policy, the deck that dominates top-rated replays (Dragapult, 57% win rate there) scores only 0.49 against the gauntlet, while a Kangaskhan/Crustle deck scores 0.73 (240 games each): the best deck for an agent depends on the agent.

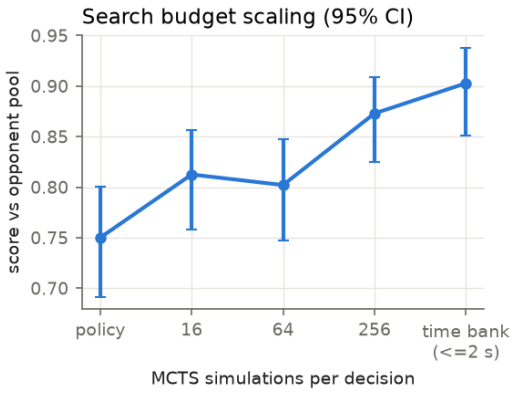


Figure 1. Score against the six-opponent gauntlet (Crustle deck, 180–240 games per point, 95% CI) as a function of search budget; "time bank" is the deployed agent (up to 2 s per decision).

Search. Search is the main source of strength (Fig. 1): 0.75 for the raw policy, 0.87 at 256 simulations, 0.90 for the deployed agent (0.93 against a reconstruction of our previous best Kaggle agent), reproduced on a second seed. Surprisingly, the learned belief gives no measurable gain when used to sample hands, and splitting a fixed budget over four determinizations is *worse* than one deep tree (-75 Elo): at these depths the tree rarely reaches situations where the opponent's exact hand matters. ExIt with 64-simulation targets does not change the raw policy (0.50) but makes the search stronger (+49 Elo), while ExIt with noisy 16-simulation targets did not help and a third round brought no further gain (0.50).

Ladder. Our previous agents sit at 650 and 500. The new agent rated $\approx$737 (4 worlds) and $\approx$768 (1 world) after 10 hours, and the ExIt-2 version reached $\approx$770 within 2 hours. Unlike the old protocol, the local ranking (1 world $>$ 4 worlds) matches the ladder.

5. Discussion and conclusions

A small imitation network is a good prior, but the gains come from looking ahead with the real game engine; more thinking time keeps helping. Better hidden-information modelling helps prediction but not play at our search depth, a negative result we confirmed with two budgets and an ablation on the number of worlds. Limitations: four of six gauntlet opponents share our network, ladder ratings are still converging, and expert iteration plateaued after two useful rounds at our compute. Next steps are more ExIt rounds with deeper targets and information-set search that shares statistics across worlds.

6. Statement on the use of AI

I used an AI assistant (Claude, Anthropic) extensively throughout the project, as a pair programmer and research assistant. It helped review the previous code base and its failure modes, propose the experimental plan, write most of the new code (replay parser, tokeniser, model and training scripts, self-play, MCTS, evaluation and statistics, submission packaging), launch and monitor experiments on my machine through a job runner, analyse results, produce the plots, and draft this report. I chose the direction of the project, reviewed the code and results, ran the Kaggle submissions, and checked that every number in this report comes from the logged experiments (`RESULTS.md` in the repository); I take full responsibility for the content and can explain all parts of it.

References

Anthony, T., Tian, Z., and Barber, D. Thinking fast and slow with deep learning and tree search. In *Advances in Neural Information Processing Systems*, 2017.

Cowling, P. I., Powley, E. J., and Whitehouse, D. Information set Monte Carlo tree search. *IEEE Transactions on Computational Intelligence and AI in Games*, 4(2): 120–143, 2012.

Long, J., Sturtevant, N. R., Buro, M., and Furtak, T. Understanding the success of perfect information Monte Carlo sampling in game tree search. In *AAAI Conference on Artificial Intelligence*, 2010.

Silver, D., Huang, A., Maddison, C. J., Guez, A., Sifre, L., et al. Mastering the game of Go with deep neural networks and tree search. *Nature*, 529(7587):484–489, 2016.

Silver, D., Hubert, T., Schrittwieser, J., Antonoglou, I., Lai, M., et al. A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play. *Science*, 362(6419):1140–1144, 2018.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017.

Vinyals, O., Babuschkin, I., Czarnecki, W. M., Mathieu, M., et al. Grandmaster level in StarCraft II using multi-agent reinforcement learning. *Nature*, 575(7782):350–354, 2019.